DATA SCIENCE

Pandas

Master DataFrames end to end — indexing, groupby, joins, reshaping, time series, I/O, and the vectorized habits that keep pandas fast.

pandas · python · data-science · dataframe · groupby · etl · analytics

Study Pack · Comprehensive reference & interview prep

Pandas is the workhorse of tabular data in Python: a fast, expressive layer over NumPy that gives you labeled, heterogeneous tables you can slice, group, join, reshape, and analyze in a handful of lines. If NumPy gives you arrays, pandas gives you a spreadsheet with a programmable brain.

What It Is and Why It Matters

Pandas provides two core data structures. A **Series** is a one-dimensional labeled array (a single column). A **DataFrame** is a two-dimensional labeled table — effectively a dictionary of Series that share a common row index. Everything you do in pandas revolves around these two objects and the *index* that labels them.

> **Mental model**
> Think of a DataFrame as columns of NumPy arrays glued together, each sharing one row index. Column operations are vectorized and fast; per-row Python loops are slow. The whole art of good pandas is pushing work into vectorized column operations.

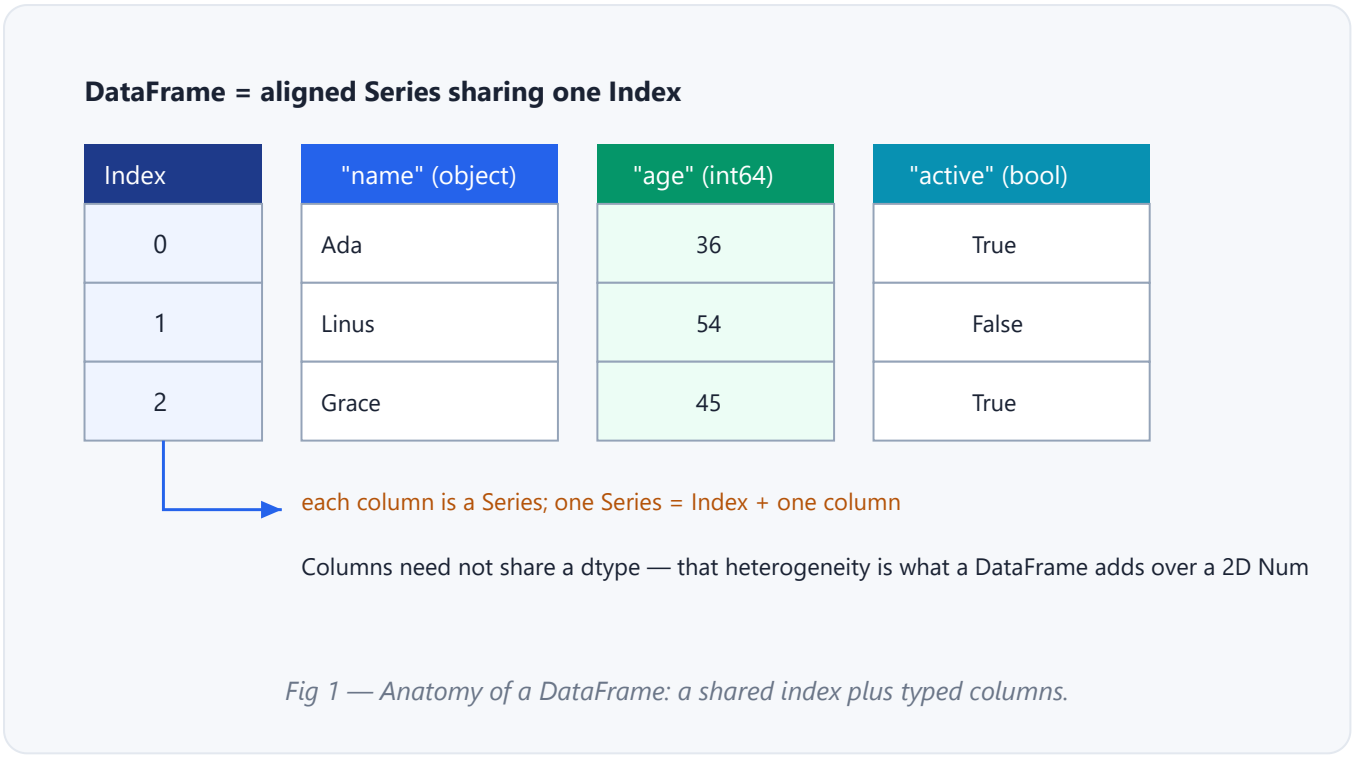


Fig 1 — Anatomy of a DataFrame: a shared index plus typed columns.

LABELED

Rows and columns carry labels, so alignment is automatic and bugs from positional mismatch disappear.

VECTORIZED

Operations run in C over whole columns — orders of magnitude faster than Python loops.

INTEROPERABLE

Reads CSV, Excel, Parquet, JSON, SQL; integrates with NumPy, matplotlib, scikit-learn, Arrow.

Setup / Getting Started

BASH

```bash
# Install (pandas 2.x is current; 2.0+ brings Arrow-backed dtypes)
pip install pandas

# Recommended extras for I/O you will actually use
pip install pyarrow openpyxl sqlalchemy   # parquet, excel, SQL

# Verify
python -c "import pandas as pd; print(pd.__version__)"
```

PYTHON

```python
import pandas as pd
import numpy as np

# Build a DataFrame from a dict of columns (most common constructor)
df = pd.DataFrame({
    "name":   ["Ada", "Linus", "Grace", "Alan"],
    "age":    [36, 54, 45, 41],
    "team":   ["A", "B", "A", "B"],
    "salary": [95000, 88000, 120000, np.nan],
})

print(df.shape)      # (4, 4)  → rows, cols
print(df.dtypes)     # per-column types
df.head(2)           # first 2 rows
df.info()            # dtypes + non-null counts + memory
df.describe()        # summary stats for numeric columns
```

✦ **Display sanity**

In notebooks, set `pd.set_option("display.max_columns", None)` and `pd.set_option("display.width", 160)` so wide tables stop wrapping awkwardly.

Series & DataFrame Essentials

A Series pairs `values` (a NumPy or Arrow-backed array) with an `index`. A DataFrame adds a `columns` index on top.

PYTHON

```python
s = pd.Series([10, 20, 30], index=["a", "b", "c"], name="score")
s["b"]            # 20  → label access
s.values          # array([10, 20, 30])
s.index           # Index(['a','b','c'], dtype='object')

# Alignment: arithmetic matches on the index, NOT position
left  = pd.Series([1, 2, 3], index=["a", "b", "c"])
right = pd.Series([10, 20],   index=["b", "c"])
left + right      # a=NaN, b=12, c=23  (a has no match → NaN)

# DataFrame column access
df["age"]         # a Series
df[["name", "age"]]   # a DataFrame (list → multiple columns)
df.age            # attribute access (only for valid identifier names)
```

> ⚠ **Columns vs rows**
>
> `df["age"]` selects a *column*. To select *rows* you need `.loc` / `.iloc` (next section). `df[0]` raises a KeyError unless a column is literally named `0`.

Dtypes you should recognize

Dtype	Means	Note
`int64` / `float64`	NumPy numeric	float64 holds NaN; int64 cannot
`object`	Python objects, usually strings	Slow; prefer `string` dtype
`string`	Dedicated string dtype	`df["c"].astype("string")`
`category`	Finite set of labels	Big memory + speed win for low-cardinality text
`datetime64[ns]`	Timestamps	Unlocks `.dt` accessor
`Int64` (capital I)	Nullable integer	Holds `pd.NA`, unlike int64
`bool` / `boolean`	Boolean / nullable boolean	Used for masks

Indexing: loc, iloc, and Selection

This is the single most important thing to get right. There are two selectors:

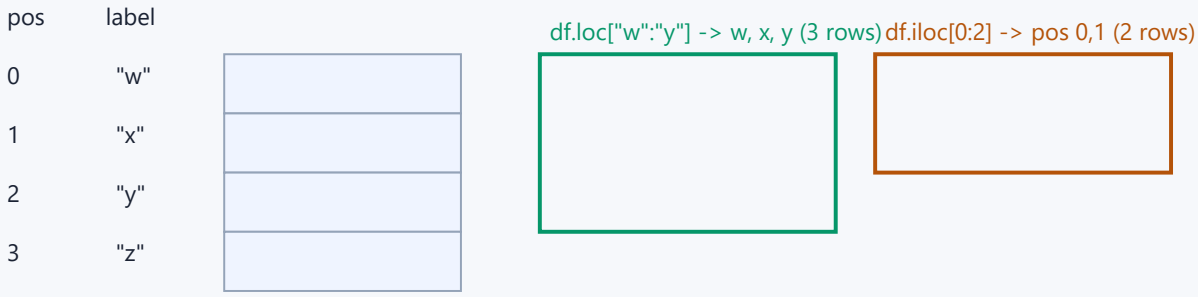
`.LOC[ROWS, COLS]`

Label-based. Endpoints are **inclusive**. Use for named rows/columns and boolean masks.

`.ILOC[ROWS, COLS]`

Integer-position based. Endpoints are **exclusive** (like Python slices). Use for positional access.

loc (labels, inclusive) vs iloc (positions, exclusive)



Same object, two coordinate systems. Mixing them is the #1 source of off-by-one bugs.

Fig 2 — Label slicing includes the endpoint; positional slicing does not.

PYTHON

```python
df = df.set_index("name")   # make 'name' the row label

# --- label-based (.loc) ---
df.loc["Ada"]                       # one row as a Series
df.loc["Ada", "age"]                # scalar
df.loc[["Ada", "Grace"], ["age", "team"]]   # subframe
df.loc["Ada":"Grace"]               # label slice, INCLUSIVE of Grace

# --- position-based (.iloc) ---
df.iloc[0]                          # first row
df.iloc[0:2]                        # first two rows, EXCLUSIVE of index 2
df.iloc[-1]                         # last row
df.iloc[0, 1]                       # row 0, column 1 by position

# --- boolean filtering (goes through .loc) ---
df.loc[df["age"] > 42]              # rows where age > 42
df.loc[(df["age"] > 42) & (df["team"] == "A")]   # & / | , parens required
df.loc[df["team"].isin(["A"])]
df.loc[df["age"].between(40, 50)]

# --- assignment through loc (no chained-indexing trap) ---
df.loc[df["age"] > 50, "team"] = "senior"
```

Chained indexing

Never assign via `df[df.age > 50]["team"] = ...`. That is *chained indexing*: it may write to a temporary copy and silently do nothing (the classic `SettingWithCopyWarning`). Always use a single `df.loc[mask, "col"] = value`.

Filtering & Transforming Columns

PYTHON

```python
# query(): string expressions, often cleaner for complex filters
df.query("age > 42 and team == 'A'")
threshold = 42
df.query("age > @threshold")          # @ references Python variables

# Create / overwrite columns
df["bonus"] = df["salary"] * 0.1
df["senior"] = df["age"] ≥ 45

# assign(): returns a new frame, chainable, no mutation
out = (df
       .assign(tax=lambda d: d["salary"] * 0.3,
               net=lambda d: d["salary"] - d["salary"] * 0.3))

# Drop / rename
df.drop(columns=["bonus"])
df.rename(columns={"salary": "pay"})
df.drop(index=["Alan"])               # drop a row by label
```

✦ **Method chaining**

Chaining with `.assign`, `.query`, `.pipe` keeps transformations readable and avoids intermediate variables. Wrap the chain in parentheses so you can put each step on its own line.

Missing Data

Missing values appear as `NaN` (float), `NaT` (datetime), or `pd.NA` (nullable dtypes). Detect, drop, or fill them.

PYTHON

```python
df.isna()                 # boolean DataFrame
df.isna().sum()           # count of missing per column  ← do this first
df["salary"].notna()

# Drop
df.dropna()                       # drop rows with ANY NaN
df.dropna(subset=["salary"])      # only consider these columns
df.dropna(axis=1, thresh=3)       # drop cols with < 3 non-null values

# Fill
df["salary"].fillna(df["salary"].median())
df.fillna({"salary": 0, "team": "unknown"})   # per-column fills
df["salary"].ffill()              # forward-fill (carry last valid)
df["salary"].bfill()              # back-fill

# Interpolate (time series / ordered numeric)
df["salary"].interpolate(method="linear")
```

> ⚠ **int64 cannot hold NaN**
>
> A column of integers with a missing value becomes `float64` automatically. If you need true nullable integers, use the `Int64` dtype: `df["n"].astype("Int64")`.

GroupBy: Split-Apply-Combine

GroupBy is pandas' most powerful idiom. You **split** rows into groups by key, **apply** a function to each group, and **combine** the results back into a Series or DataFrame.

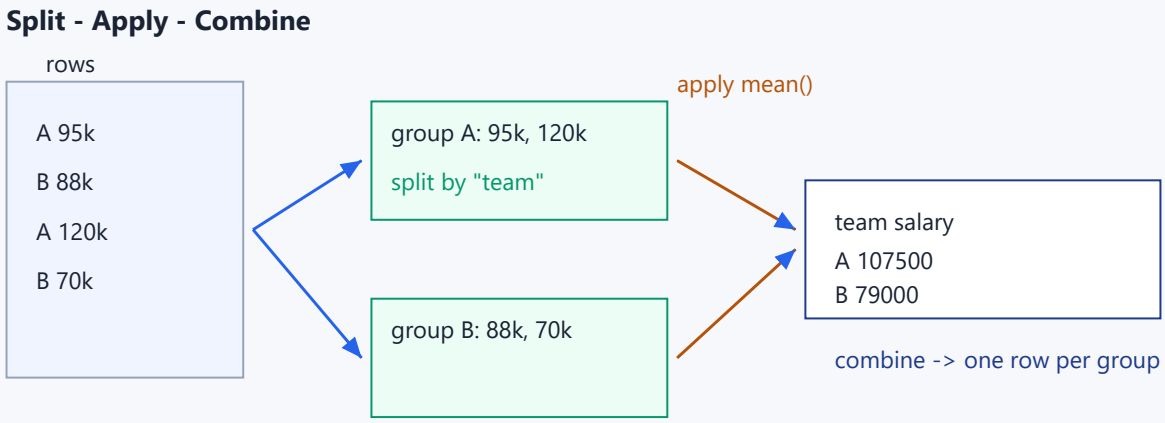


Fig 3 — GroupBy partitions rows by key, reduces each partition, and stitches results together.

PYTHON

```python
# Single aggregation
df.groupby("team")["salary"].mean()

# Multiple columns, multiple functions via agg()
df.groupby("team").agg(
    avg_pay=("salary", "mean"),
    max_age=("age", "max"),
    n=("salary", "size"),
)   # named aggregation → clean, flat column names

# Group by several keys → MultiIndex result
df.groupby(["team", "senior"])["salary"].sum()

# transform(): broadcast group result back to original shape (same length)
df["pay_vs_team_avg"] = df["salary"] - df.groupby("team")["salary"].transform("mean")

# filter(): keep only groups meeting a condition
df.groupby("team").filter(lambda g: g["salary"].mean() > 80000)

# apply(): most flexible, slowest — use when agg/transform cannot express it
df.groupby("team").apply(lambda g: g.nlargest(2, "salary"))
```

✦ **agg vs transform**

Use `agg` when you want one row per group (a summary). Use `transform` when you want a result aligned to the original rows (e.g. group mean on every row). Reach for `apply` only when neither fits — it is the slow fallback.

Merge, Join & Concatenate

Combine datasets by key (`merge`), by index (`join`), or by stacking (`concat`).

PYTHON

```python
employees = pd.DataFrame({"emp_id": [1, 2, 3], "name": ["Ada", "Linus", "Grace"], "dep
depts     = pd.DataFrame({"dept_id": [10, 20, 30], "dept": ["Eng", "Sales", "Ops"]})

# INNER (default): only matching keys
pd.merge(employees, depts, on="dept_id", how="inner")

# LEFT: keep all left rows, NaN where no match
pd.merge(employees, depts, on="dept_id", how="left")

# Different key names on each side
pd.merge(a, b, left_on="country_code", right_on="iso", how="left")

# Stacking
pd.concat([df1, df2], axis=0)             # stack rows (union)
pd.concat([df1, df2], axis=1)             # stack columns (align on index)

# validate guards against accidental fan-out / dup keys
pd.merge(employees, depts, on="dept_id", how="left", validate="many_to_one")
```

how	Keeps	SQL analog
`inner`	Only keys in both	INNER JOIN
`left`	All left + matches	LEFT JOIN
`right`	All right + matches	RIGHT JOIN
`outer`	Union of keys	FULL OUTER JOIN
`cross`	Cartesian product	CROSS JOIN

⚠ **Silent row explosion**
If the right side has duplicate keys, a left merge multiplies rows (a many-to-many fan-out). Pass `validate="many_to_one"` or `"one_to_one"` to make pandas raise instead of silently inflating your data.

Reshaping: pivot, melt, stack

Data arrives either *wide* (one column per variable) or *long* (one row per observation). Reshape between them.

PYTHON

```python
# Long → wide: pivot_table (aggregates duplicates safely)
sales = pd.DataFrame({
    "date": ["2026-01", "2026-01", "2026-02", "2026-02"],
    "region": ["N", "S", "N", "S"],
    "revenue": [100, 80, 120, 95],
})
wide = sales.pivot_table(index="date", columns="region",
                         values="revenue", aggfunc="sum", fill_value=0)

# Wide → long: melt
long = wide.reset_index().melt(id_vars="date",
                               var_name="region", value_name="revenue")

# stack / unstack move an index level between rows and columns
multi = df.groupby(["team", "senior"])["salary"].sum()
multi.unstack()          # 'senior' level → columns
multi.unstack().stack()  # back to long

# crosstab: frequency table
pd.crosstab(df["team"], df["senior"])
```

> **pivot vs pivot_table**
>
> `pivot` fails if there are duplicate index/column pairs. `pivot_table` aggregates them via `aggfunc` — use it as your default and reserve `pivot` for guaranteed-unique data.

apply, map & Vectorization

PYTHON

```python
# map(): element-wise on a Series (dict lookup or function)
df["team_full"] = df["team"].map({"A": "Alpha", "B": "Bravo"})

# apply() on a Series: element-wise function
df["name_len"] = df.index.to_series().apply(len)

# apply() on a DataFrame: axis=1 runs per row (SLOW — avoid in hot paths)
df["flag"] = df.apply(lambda r: "hi" if r["salary"] > 100000 else "lo", axis=1)

# Prefer vectorized / np.where for the common case
df["flag"] = np.where(df["salary"] > 100000, "hi", "lo")
df["bucket"] = np.select(
    [df["age"] < 40, df["age"] < 50],
    ["junior", "mid"],
    default="senior",
)

# String operations are vectorized via .str
df["name"].str.upper()
df["name"].str.contains("a", case=False)
df["name"].str.replace("Ada", "ADA", regex=False)
```

apply(axis=1) is a loop in disguise

It calls your Python function once per row. On a million rows it can be 100x slower than the vectorized equivalent. Reach for `np.where`, `np.select`, `.str`, `.dt`, or boolean masks first.

Datetime & Time Series

PYTHON

```python
# Parse strings to datetime64
df["ts"] = pd.to_datetime(df["ts"], format="%Y-%m-%d", errors="coerce")

# The .dt accessor exposes calendar fields
df["ts"].dt.year
df["ts"].dt.month_name()
df["ts"].dt.dayofweek          # Monday=0
df["ts"].dt.quarter

# Set a DatetimeIndex to unlock resampling & partial-string slicing
ts = df.set_index("ts").sort_index()
ts.loc["2026-01"]              # all rows in January 2026
ts.loc["2026-01-01":"2026-03-31"]

# Resample: groupby over time buckets
ts["revenue"].resample("ME").sum()     # month-end totals
ts["revenue"].resample("W").mean()      # weekly average

# Rolling windows
ts["revenue"].rolling(window=7).mean()  # 7-period moving average
ts["revenue"].shift(1)                  # lag by one period
ts["revenue"].pct_change()              # period-over-period % change

# Timezones
ts.tz_localize("UTC").tz_convert("Asia/Kolkata")
```

✦ **Always sort a time index**

Partial-string slicing and `rolling` assume monotonic order. Call `.sort_index()` right after you set a DatetimeIndex.

Reading & Writing: CSV, Excel, SQL, Parquet

PYTHON

```python
# CSV — the daily driver
df = pd.read_csv("data.csv",
                 parse_dates=["date"],       # parse while reading
                 dtype={"zip": "string"},     # keep leading zeros
                 usecols=["date", "zip", "amount"],  # read only what you need
                 na_values=["", "NA", "null"])
df.to_csv("out.csv", index=False)

# Excel (needs openpyxl)
pd.read_excel("book.xlsx", sheet_name="Q1")
df.to_excel("out.xlsx", sheet_name="results", index=False)

# SQL (needs SQLAlchemy engine)
from sqlalchemy import create_engine
engine = create_engine("postgresql+psycopg2://user:pw@host/db")
pd.read_sql("SELECT * FROM sales WHERE year = 2026", engine)
df.to_sql("results", engine, if_exists="replace", index=False)

# Parquet — columnar, typed, compressed; best for analytics pipelines
df.to_parquet("data.parquet")            # needs pyarrow
pd.read_parquet("data.parquet", columns=["date", "amount"])

# JSON
pd.read_json("data.json")
pd.json_normalize(nested_records)        # flatten nested JSON into columns
```

> **Prefer Parquet for pipelines**
> CSV loses dtypes, has no compression, and reparses on every read. Parquet stores dtypes, compresses well, and lets you read only the columns you need — often 5-20x faster and smaller.

Mini Worked Example: Monthly Revenue Report

A realistic end-to-end chain: read raw orders, clean, enrich, aggregate, and export.

PYTHON

```python
import pandas as pd

orders = pd.read_csv("orders.csv", parse_dates=["order_date"])
customers = pd.read_csv("customers.csv")   # customer_id, region, tier

report = (
    orders
      .dropna(subset=["amount"])                      # drop incomplete rows
      .assign(month=lambda d: d["order_date"].dt.to_period("M"))
      .merge(customers, on="customer_id", how="left",  # enrich with region
             validate="many_to_one")
      .groupby(["month", "region"], as_index=False)    # split by month+region
      .agg(revenue=("amount", "sum"),
           orders=("amount", "size"),
           avg_order=("amount", "mean"))
      .sort_values(["month", "revenue"], ascending=[True, False])
)

report["revenue_share"] = (
    report["revenue"] / report.groupby("month")["revenue"].transform("sum")
)
report.to_parquet("monthly_revenue.parquet", index=False)
print(report.head())
```

Show the transformation step by step

Cheat-Sheet Reference

Task	Code
Shape / columns	`df.shape` , `df.columns`
First / last n	`df.head(n)` , `df.tail(n)`
Random sample	`df.sample(5)`
Select column	`df["c"]`
Select rows by label	`df.loc[mask]`
Select rows by position	`df.iloc[0:5]`
Filter	`df.query("a > 1")`
New column	`df.assign(c= ... )`
Sort	`df.sort_values("c", ascending=False)`
Unique / counts	`df["c"].unique()` , `df["c"].value_counts()`
Aggregate by group	`df.groupby("k").agg( ... )`
Join	`pd.merge(a, b, on="k", how="left")`
Pivot	`df.pivot_table(index=, columns=, values=)`
Missing count	`df.isna().sum()`
Rename	`df.rename(columns={"a": "b"})`
Reset / set index	`df.reset_index()` , `df.set_index("c")`
Apply vectorized bins	`pd.cut(df["c"], bins=[0,10,20])`
Memory usage	`df.memory_usage(deep=True)`

Pandas vs SQL vs Excel

Concept	Pandas	SQL
Filter rows	`df.query("x > 1")`	`WHERE x > 1`
Pick columns	`df[["a","b"]]`	`SELECT a, b`
Aggregate	`groupby().agg()`	`GROUP BY`
Join	`merge(how="left")`	`LEFT JOIN`
Sort	`sort_values()`	`ORDER BY`
Distinct	`drop_duplicates()`	`DISTINCT`
Top-N per group	`groupby().head(n)`	`ROW_NUMBER() ≤ n`

Performance Tips

VECTORIZE EVERYTHING

Replace `apply(axis=1)` and Python loops with column arithmetic, `np.where`, `np.select`, and `.str` / `.dt` accessors.

SHRINK DTYPES

Convert low-cardinality strings to `category` and downcast numerics (`pd.to_numeric(s, downcast="integer")`). Big memory wins.

READ LESS

Use `usecols`, `dtype`, and `nrows` on read. Prefer Parquet so you only load needed columns.

AVOID GROWING FRAMES

Never `concat` inside a loop. Collect pieces in a list and `pd.concat` once at the end.

CHUNK BIG FILES

`pd.read_csv(path, chunksize=100_000)` yields chunks you can process and reduce without loading everything.

KNOW WHEN TO LEAVE

At tens of GB, consider Polars, DuckDB, or Dask. Pandas is single-machine and largely single-threaded.

PYTHON

```python
# Measure before optimizing
%timeit df.apply(lambda r: r.a + r.b, axis=1)   # slow (row loop)
%timeit df["a"] + df["b"]                         # fast (vectorized)

# Category dtype can cut memory of text columns by 10x+
before = df.memory_usage(deep=True).sum()
df["team"] = df["team"].astype("category")
after = df.memory_usage(deep=True).sum()

# Build once, not in a loop
pieces = [process(chunk) for chunk in pd.read_csv("big.csv", chunksize=100_000)]
result = pd.concat(pieces, ignore_index=True)
```

✓ **Rule of thumb**

If you wrote a Python `for` loop over rows, there is almost always a vectorized pandas/NumPy expression that is both shorter and dramatically faster. Look for it first.

Best Practices & Common Pitfalls

✓ **Do**

Use `.loc[mask, col] = value` for conditional assignment; chain with `.assign` / `.query` / `.pipe`; set explicit dtypes on read; profile with `df.info()` and `%timeit`; keep a tidy (long) layout for analysis.

⚠ **Watch out**

`inplace=True` rarely saves memory and often hurts readability — prefer reassignment. Comparing floats with `==` is unreliable; use `np.isclose`. `df.append` was removed in pandas 2.0 — use `pd.concat`.

✕ **Classic bugs**

(1) Chained-indexing assignment silently fails. (2) Merging on a non-unique key explodes row counts. (3) Forgetting that integer columns coerce to float on missing data. (4) Using Python `and` / `or` instead of `&` / `|` in masks (and forgetting parentheses).

Interview Questions & Answers

What is the difference between `.loc` and `.iloc`?

`.loc` is label-based and its slice endpoints are inclusive; `.iloc` is integer-position based and its endpoints are exclusive, like Python slicing. Use `.loc` with named labels and boolean masks, `.iloc` for positional access. Mixing them is a common off-by-one source.

Explain split-apply-combine and when you'd use transform vs agg.

GroupBy splits rows into groups by key, applies a function to each group, and combines results. `agg` reduces each group to one row (a summary). `transform` returns a result the same length as the input, broadcasting the group result back to every row — ideal for things like subtracting a per-group mean. `apply` is the flexible but slow fallback.

What is the `SettingWithCopyWarning` and how do you avoid it?

It warns that you may be assigning to a temporary copy produced by chained indexing (`df[mask]["col"] = x`), so the write might not land on the original frame. Fix it by doing a single indexed assignment: `df.loc[mask, "col"] = x`.

Why does an integer column become float after you introduce a missing value?

NumPy's `int64` has no representation for NaN, so pandas upcasts the column to `float64` to store the missing value. To keep integers, use the nullable `Int64` dtype, which supports `pd.NA`.

How do merge types differ, and what causes a row explosion in a join?

`inner` keeps only matching keys, `left` / `right` keep all rows from one side, `outer` keeps the union. A row explosion happens when the key is non-unique on the other side, creating a many-to-many fan-out. Guard with the `validate` parameter (e.g. `"many_to_one"`).

When should you avoid `apply(axis=1)`?

Almost always in performance-sensitive code: it calls a Python function per row, defeating vectorization and running 10-100x slower. Prefer vectorized arithmetic, `np.where`, `np.select`, boolean masks, or the `.str` / `.dt` accessors.

How do you reshape between wide and long formats?

`pivot_table` (or `pivot`) goes long-to-wide, aggregating duplicates via `aggfunc`; `melt` goes wide-to-long. `stack` / `unstack` move levels between the row and column indexes. Prefer `pivot_table` because `pivot` errors on duplicate index/column pairs.

What does `groupby().transform("mean")` return versus `groupby().mean()`?

`mean()` returns one value per group (shape = number of groups). `transform("mean")` returns a Series the same length as the original, with each row filled by its group's mean — useful for creating per-row derived features like deviation from the group average.

How would you reduce the memory footprint of a large DataFrame?

Convert low-cardinality text columns to `category`, downcast numerics with `pd.to_numeric( ... , downcast= ... )`, read only needed columns with `usecols`, set explicit dtypes on read, and store intermediate data as Parquet. Measure with `df.memory_usage(deep=True)`.

What is index alignment and how does it affect arithmetic?

Pandas aligns operands on their index labels (not position) before arithmetic. Labels present in one operand but not the other produce `NaN`. This prevents silent position-based mismatches but surprises people who expect positional behavior — use `.values` or `.reset_index(drop=True)` to force positional math.

How do you handle a CSV with dates and leading-zero zip codes?

Pass `parse_dates=["date"]` so dates become `datetime64`, and `dtype={"zip": "string"}` so the zip stays text and keeps leading zeros (reading it as a number would strip them). Also set `na_values` to catch the file's missing-value sentinels.

When would you choose Polars or DuckDB over pandas?

When data exceeds comfortable single-machine memory, when you need multi-core/parallel execution, or when you want lazy query optimization. Pandas is single-machine and mostly single-threaded; Polars (Rust, multi-threaded, lazy) and DuckDB (in-process SQL on larger-than-memory data) scale better while interoperating with pandas via Arrow.

Further Learning

- **Official docs** — the "10 minutes to pandas" and "Essential basic functionality" guides; the API reference is excellent for discovering parameters.
- **Tidy data** — Hadley Wickham's paper; the long-format discipline makes groupby/pivot natural.
- **NumPy fundamentals** — broadcasting and dtypes underpin pandas performance.
- **Arrow & Parquet** — the modern columnar backbone; learn `pyarrow` dtypes in pandas 2.x.
- **Polars & DuckDB** — the next step when pandas hits scale limits; both read/write Arrow and Parquet.
- **Practice** — rebuild a SQL analytics report entirely in pandas, then profile and vectorize the slow parts.